

```
import pandas as pd
```

```
df1 = pd.read_csv(r'C:\Users\LENOVO\Downloads\game sales - game  
sales.csv')
```

```
df1
```

	GameID	Platform	SalesRegion	UnitsSold	Price
0	1	Xbox	North America	861216	45.39
1	2	PlayStation	North America	444626	53.55
2	3	PlayStation	North America	569637	34.20
3	4	PlayStation	South America	977779	43.90
4	5	PC	South America	473349	21.61
...	...	...	...	...	...
145	146	Xbox	South America	795629	24.76
146	147	PC	North America	548237	40.10
147	148	PlayStation	Asia	120934	55.72
148	149	Xbox	Asia	161486	52.82
149	150	Nintendo Switch	North America	806262	28.67

```
[150 rows x 5 columns]
```

```
df2 = pd.read_csv(r'C:\Users\LENOVO\Downloads\Games - Games.csv')
```

```
df2
```

	GameID	GameTitle	Genre
ReleaseDate \			
0	1	Right-sized human-resource forecast	Adventure
2024-02-24			
1	2	Optional tertiary frame	Simulation
2021-11-02			
2	3	Team-oriented static standardization	Simulation
2022-08-16			
3	4	Inverse mobile database	Strategy
2024-03-15			
4	5	Inverse mission-critical array	Adventure
2022-05-08			
..	...	...	...
...			
145	146	Multi-layered eco-centric approach	Puzzle
2024-06-19			
146	147	Pre-emptive contextually-based moderator	Sci-Fi
2024-08-10			
147	148	Cross-group 24/7 benchmark	Strategy
2023-05-30			
148	149	Organized dynamic moderator	Strategy
2022-09-08			
149	150	Front-line regional hierarchy	Strategy
2023-01-18			

```
Developer
```

```

0      Pixel Studios
1      StarGame Studios
2      Pixel Studios
3      Sunrise Interactive
4      Pixel Studios
..
145     DarkMoon Games
146     DarkMoon Games
147     SeaWave Games
148     Sunrise Interactive
149     SeaWave Games

```

```
[150 rows x 5 columns]
```

```

joined_df = pd.merge(df1,df2,on='GameID', how='inner')
joined_df

```

	GameID	Platform	SalesRegion	UnitsSold	Price \
0	1	Xbox	North America	861216	45.39
1	2	PlayStation	North America	444626	53.55
2	3	PlayStation	North America	569637	34.20
3	4	PlayStation	South America	977779	43.90
4	5	PC	South America	473349	21.61
..	...	...	...	...	...
145	146	Xbox	South America	795629	24.76
146	147	PC	North America	548237	40.10
147	148	PlayStation	Asia	120934	55.72
148	149	Xbox	Asia	161486	52.82
149	150	Nintendo Switch	North America	806262	28.67

	GameTitle	Genre	ReleaseDate
0	Right-sized human-resource forecast	Adventure	2024-02-24
1	Optional tertiary frame	Simulation	2021-11-02
2	Team-oriented static standardization	Simulation	2022-08-16
3	Inverse mobile database	Strategy	2024-03-15
4	Inverse mission-critical array	Adventure	2022-05-08
..	...	...	...
145	Multi-layered eco-centric approach	Puzzle	2024-06-19
146	Pre-emptive contextually-based moderator	Sci-Fi	2024-08-10
147	Cross-group 24/7 benchmark	Strategy	2023-05-30
148	Organized dynamic moderator	Strategy	2022-09-08

```

Developer
0      Pixel Studios
1      StarGame Studios
2      Pixel Studios
3      Sunrise Interactive
4      Pixel Studios
..
145     DarkMoon Games
146     DarkMoon Games
147     SeaWave Games
148     Sunrise Interactive
149     SeaWave Games

```

```
[150 rows x 9 columns]
```

```
joined_df.shape
```

```
(150, 9)
```

```
joined_df = joined_df.dropna()
```

```
len(joined_df)
```

```
150
```

```
joined_df.isnull().sum()
```

```

GameID      0
Platform    0
SalesRegion 0
UnitsSold   0
Price       0
GameTitle   0
Genre       0
ReleaseDate 0
Developer   0
dtype: int64

```

```
joined_df[joined_df.isnull().any(axis=1)]
```

```
Empty DataFrame
```

```
Columns: [GameID, Platform, SalesRegion, UnitsSold, Price, GameTitle, Genre, ReleaseDate, Developer]
```

```
Index: []
```

```
joined_df['Price']
```

```

0      45.39
1      53.55
2      34.20
3      43.90
4      21.61
...
145    24.76
146    40.10
147    55.72
148    52.82
149    28.67
Name: Price, Length: 150, dtype: float64

```

```

# sampling
try:
    import numpy as np

    sample_mean_value = []

    for i in range(0,1001):
        sample = np.random.choice(joined_df['Price'], size=30,
replace=True, p=None)
        sample_mean_value.append(sample.mean())

    sample_mean_value = [round(float(z),2) for z in sample_mean_value]
    print(sample_mean_value)

except Exception as e:
    print(f'Some error occured {e}')

```

```

[35.56, 41.37, 38.67, 37.31, 35.33, 41.45, 38.68, 41.82, 38.02, 38.25,
38.5, 38.15, 41.0, 39.41, 38.42, 38.89, 40.26, 38.34, 35.03, 38.58,
39.53, 41.03, 41.23, 37.84, 40.75, 39.41, 39.68, 37.49, 33.99, 37.49,
36.87, 38.43, 36.18, 34.95, 35.71, 40.34, 39.65, 35.82, 39.32, 35.89,
37.3, 38.23, 37.16, 40.65, 35.95, 37.26, 38.61, 40.8, 39.09, 36.98,
38.51, 37.95, 38.78, 36.8, 39.13, 36.3, 40.12, 36.45, 39.45, 36.25,
38.42, 35.34, 37.24, 39.04, 40.58, 36.25, 38.42, 39.28, 36.7, 35.28,
37.9, 40.8, 39.48, 40.41, 37.48, 40.32, 40.83, 39.55, 35.95, 39.23,
37.71, 38.41, 36.92, 39.26, 39.89, 40.04, 37.93, 41.45, 40.47, 36.56,
39.44, 40.67, 39.38, 37.97, 37.02, 37.6, 39.0, 39.53, 39.18, 39.24,
37.64, 42.26, 41.31, 39.09, 36.75, 37.16, 38.71, 38.88, 39.84, 33.05,
40.39, 39.33, 39.19, 38.65, 39.44, 38.78, 38.64, 40.56, 38.89, 39.28,
35.99, 37.55, 43.2, 41.9, 41.04, 42.07, 34.77, 37.94, 36.22, 35.0,
38.67, 41.75, 39.46, 38.66, 41.58, 41.17, 35.97, 39.37, 36.43, 37.21,
39.33, 40.86, 38.76, 40.19, 38.34, 39.66, 35.81, 38.6, 36.98, 40.49,
36.1, 36.87, 39.93, 40.68, 38.23, 38.44, 40.73, 37.06, 39.98, 35.75,
39.93, 39.02, 40.52, 39.54, 37.44, 36.72, 40.67, 37.59, 39.0, 39.64,
41.34, 39.18, 36.97, 38.34, 37.38, 38.62, 35.03, 37.87, 39.42, 34.86,
40.2, 38.65, 38.96, 35.14, 37.21, 40.05, 38.94, 39.82, 35.13, 41.42,
39.98, 37.6, 37.38, 38.12, 39.12, 40.77, 39.16, 37.62, 36.4, 35.37,

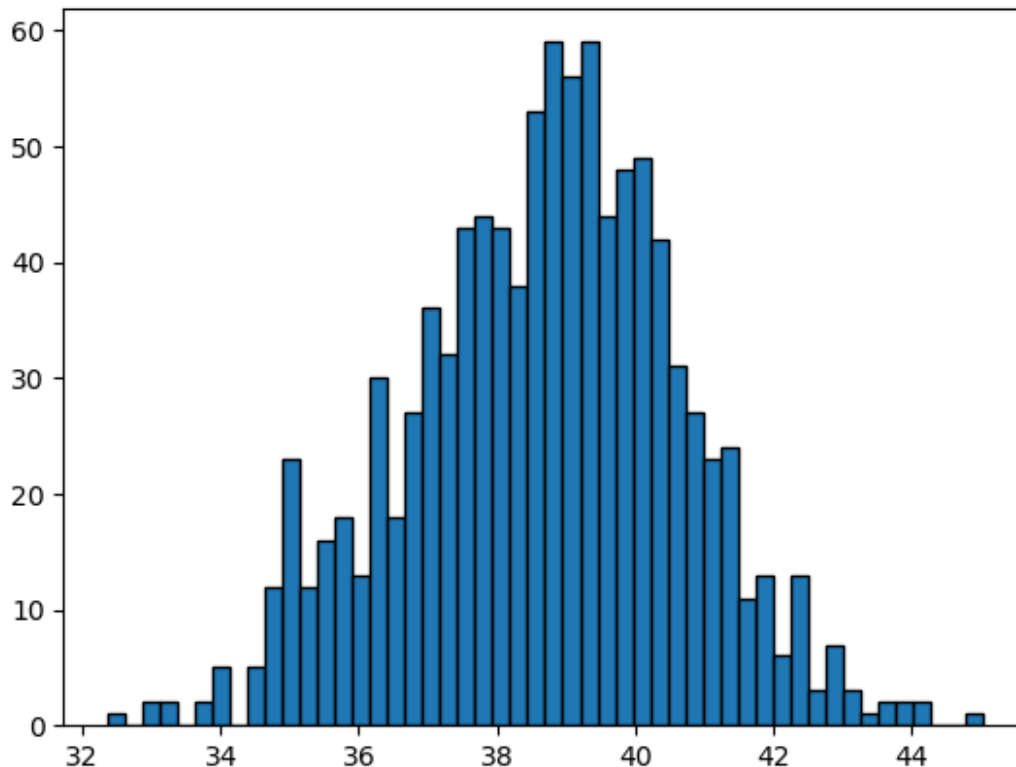
```

39.76, 34.87, 38.47, 37.56, 33.13, 39.22, 37.79, 39.89, 39.89, 38.94,
33.77, 38.75, 39.18, 37.99, 38.22, 37.34, 37.06, 40.95, 41.38, 37.06,
35.82, 43.6, 39.5, 40.55, 40.48, 38.04, 38.28, 37.58, 40.2, 38.98,
39.03, 37.78, 36.56, 42.39, 38.1, 35.65, 41.67, 34.89, 37.55, 40.12,
37.47, 36.38, 38.96, 42.66, 42.32, 38.19, 37.38, 40.9, 37.54, 37.37,
39.37, 34.65, 39.32, 34.87, 41.06, 41.13, 39.27, 38.04, 42.41, 38.83,
39.33, 37.94, 39.08, 42.81, 35.69, 37.48, 39.24, 38.77, 36.59, 35.26,
35.14, 37.34, 39.84, 38.79, 39.64, 38.09, 37.8, 38.59, 39.01, 42.9,
36.08, 35.1, 37.6, 42.27, 38.72, 35.77, 39.07, 35.42, 38.02, 34.99,
37.49, 38.07, 37.17, 39.79, 41.57, 41.24, 38.49, 36.29, 38.27, 37.27,
37.6, 40.0, 37.52, 35.03, 39.04, 37.45, 41.03, 42.45, 39.66, 37.72,
41.04, 39.08, 37.7, 38.29, 37.72, 42.4, 36.66, 39.25, 35.73, 39.04,
40.32, 37.74, 39.18, 37.88, 38.73, 39.73, 36.29, 38.56, 37.2, 44.12,
37.59, 35.58, 38.78, 39.87, 39.42, 37.76, 37.8, 37.51, 37.31, 37.85,
38.02, 38.64, 38.44, 37.15, 36.94, 39.9, 39.4, 41.14, 40.07, 39.92,
39.17, 36.2, 39.81, 39.2, 35.52, 35.55, 39.63, 42.8, 39.6, 40.45,
38.51, 35.06, 38.62, 36.3, 38.77, 40.47, 40.77, 39.38, 36.9, 40.33,
34.76, 38.61, 40.25, 39.06, 44.07, 39.79, 38.85, 38.9, 33.92, 36.49,
35.89, 35.11, 36.58, 38.08, 42.81, 39.97, 38.9, 40.08, 40.3, 39.36,
38.55, 39.55, 35.39, 37.48, 38.34, 37.68, 38.93, 35.14, 40.87, 35.61,
39.31, 36.8, 41.15, 41.02, 40.04, 40.15, 39.42, 35.53, 40.45, 36.43,
41.43, 34.56, 35.01, 39.21, 34.81, 36.8, 36.97, 38.49, 36.8, 40.18,
38.73, 43.92, 36.1, 39.1, 39.61, 34.76, 39.17, 38.53, 40.35, 42.43,
39.84, 39.42, 36.86, 37.17, 35.25, 37.29, 39.48, 40.73, 36.55, 41.83,
38.02, 37.54, 38.74, 35.13, 39.21, 39.06, 38.1, 40.27, 39.82, 36.74,
38.5, 38.25, 37.86, 32.36, 37.6, 38.91, 38.68, 39.79, 42.47, 36.8,
40.42, 39.04, 37.73, 38.28, 41.12, 43.23, 40.45, 38.42, 39.34, 37.92,
38.2, 35.66, 40.88, 39.1, 39.46, 37.7, 40.21, 35.22, 41.45, 41.35,
38.24, 40.75, 36.97, 39.2, 39.16, 37.36, 38.64, 39.27, 39.13, 37.93,
36.7, 37.95, 39.27, 35.76, 39.14, 34.72, 41.55, 37.92, 42.03, 39.86,
39.88, 37.07, 37.04, 38.54, 40.04, 40.64, 39.42, 34.6, 36.7, 36.22,
39.55, 32.97, 37.62, 38.28, 34.03, 41.43, 41.37, 39.2, 39.15, 37.16,
38.89, 40.53, 41.34, 40.61, 37.64, 37.81, 39.87, 37.77, 36.93, 40.73,
36.57, 37.65, 36.75, 38.93, 39.9, 37.28, 39.51, 40.13, 38.39, 42.64,
40.43, 40.12, 36.98, 38.31, 39.82, 34.95, 41.9, 40.65, 38.93, 38.9,
35.53, 39.49, 39.52, 41.35, 38.48, 37.74, 38.12, 39.91, 40.65, 40.21,
37.92, 38.0, 37.95, 37.0, 36.24, 41.71, 39.27, 39.18, 35.54, 39.92,
36.86, 40.08, 41.95, 36.23, 40.45, 39.78, 42.84, 36.95, 39.51, 40.22,
40.1, 39.43, 39.6, 36.37, 40.16, 37.6, 36.79, 38.75, 39.79, 40.05,
38.95, 37.52, 36.27, 38.62, 40.24, 39.06, 40.53, 40.09, 39.33, 37.89,
40.06, 42.96, 35.36, 41.34, 38.19, 36.7, 37.77, 38.88, 38.81, 40.18,
39.0, 34.13, 40.97, 38.98, 39.68, 38.74, 39.99, 38.62, 38.74, 38.08,
37.0, 39.73, 37.02, 35.52, 38.56, 38.51, 39.49, 39.4, 37.19, 37.74,
35.64, 39.89, 39.27, 40.7, 37.13, 37.52, 39.62, 38.83, 35.44, 42.28,
38.82, 38.85, 41.64, 42.3, 38.32, 37.32, 37.93, 40.67, 35.52, 39.3,
36.36, 41.78, 40.22, 38.45, 39.65, 40.58, 36.85, 38.65, 35.27, 36.23,
40.28, 37.41, 40.65, 43.27, 39.45, 36.82, 40.21, 39.71, 42.02, 39.16,
39.21, 41.82, 40.47, 33.17, 38.27, 35.52, 37.92, 41.95, 40.71, 39.73,
41.06, 40.27, 39.44, 40.23, 39.59, 41.34, 36.25, 40.79, 38.4, 39.74,

```
40.11, 38.04, 34.06, 37.31, 38.24, 37.8, 38.91, 36.65, 43.6, 39.41,
39.5, 35.37, 38.5, 37.41, 40.09, 41.56, 37.46, 36.56, 38.66, 43.09,
36.51, 38.7, 39.72, 39.81, 39.52, 40.43, 37.88, 40.43, 41.46, 39.5,
37.83, 40.29, 37.02, 39.5, 38.12, 34.95, 40.37, 41.18, 39.35, 38.6,
34.83, 39.25, 39.61, 35.05, 37.08, 37.97, 38.08, 39.78, 40.25, 38.06,
34.96, 39.85, 40.78, 38.5, 36.46, 38.8, 41.86, 37.55, 35.86, 37.37,
39.57, 37.87, 38.32, 40.65, 36.46, 36.48, 38.04, 34.87, 40.91, 40.26,
37.07, 40.24, 41.33, 40.37, 37.33, 37.38, 40.67, 41.18, 36.36, 39.11,
36.89, 34.63, 39.18, 37.97, 36.29, 38.66, 41.64, 39.72, 39.7, 36.26,
40.7, 35.77, 40.92, 40.24, 38.11, 38.92, 42.66, 40.64, 37.06, 39.11,
38.81, 39.43, 34.4, 37.56, 37.99, 41.34, 38.27, 38.08, 41.14, 39.61,
39.51, 38.74, 40.27, 38.57, 35.98, 40.84, 39.3, 41.29, 37.26, 39.9,
36.34, 37.09, 38.34, 37.16, 34.64, 40.07, 37.04, 38.57, 38.92, 38.61,
39.15, 40.0, 38.18, 37.86, 38.7, 40.68, 35.42, 39.97, 41.06, 40.94,
35.13, 41.07, 41.66, 36.27, 40.87, 41.46, 39.31, 36.01, 38.54, 38.44,
37.03, 37.27, 37.04, 37.75, 38.8, 38.37, 36.37, 39.7, 40.23, 35.95,
36.26, 39.41, 39.98, 38.13, 39.72, 37.86, 38.73, 40.17, 41.18, 38.57,
40.38, 43.82, 36.11, 40.47, 40.04, 40.31, 39.26, 37.52, 36.11, 40.2,
38.67, 38.46, 40.77, 39.24, 39.13, 37.52, 39.73, 40.57, 38.91, 38.69,
41.16, 39.68, 37.34, 40.56, 37.48, 36.41, 38.81, 38.02, 37.96, 38.8,
38.2, 39.33, 39.16, 40.09, 35.78, 34.97, 40.63, 36.85, 37.88, 40.31,
40.01, 38.77, 36.18, 40.22, 34.97, 37.07, 37.89, 40.11, 41.87, 37.58,
39.76, 40.0, 37.61, 39.07, 35.0, 39.16, 39.74, 38.17, 40.52, 35.75,
37.77, 36.71, 42.48, 38.48, 39.81, 41.75, 41.72, 39.04, 39.91, 42.1,
38.87, 40.45, 37.88, 37.02, 35.93, 35.72, 35.32, 40.89, 40.8, 41.15,
35.9, 40.62, 38.12, 42.2, 37.5, 38.35, 38.09, 39.83, 42.13, 37.8,
40.32, 38.8, 38.03, 38.6, 38.85, 39.77, 40.0, 42.27, 36.83, 40.12,
39.06, 39.27, 38.76, 40.73, 33.65, 39.14, 41.83, 45.04, 38.85, 37.27,
39.65, 39.27, 41.2, 41.27, 38.89, 41.46, 40.46, 39.53, 38.49, 38.51,
38.9, 36.3, 37.63, 40.05, 38.9, 37.81, 39.85, 39.15, 37.27, 39.13,
36.77, 39.66, 37.34, 36.25, 41.68, 42.81, 38.38, 36.46, 39.65, 39.74,
38.62]
```

```
import matplotlib.pyplot as plt
```

```
plt.hist(sample_mean_value, bins=50, edgecolor='black')
plt.show()
```



##Now if I take less sample yyou will not see a bell curve like this

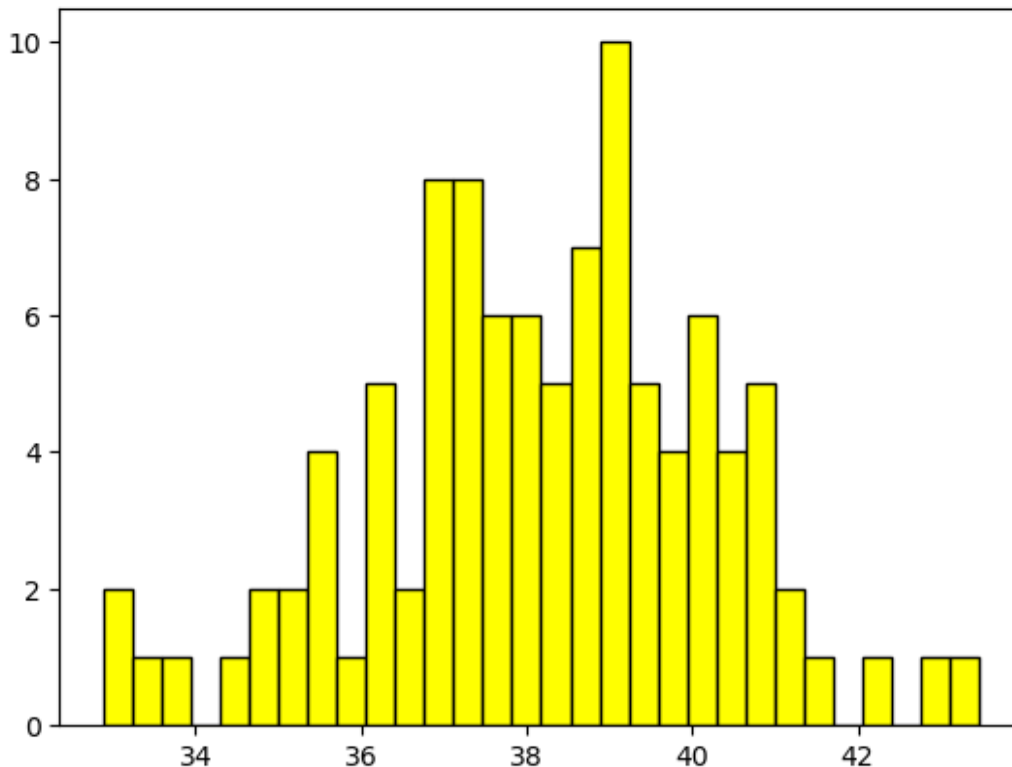
```
sample_values2 = []
```

```
for i in range(0,101):
    sample2 = np.random.choice(joined_df['Price'], size=30,
    replace=True, p=None)
    sample_values2.append(sample2.mean())
```

```
sample_values2 = [round(float(z),2) for z in sample_values2]
print(sample_values2)
```

```
[37.64, 38.59, 35.18, 39.94, 37.41, 34.4, 33.14, 38.99, 40.15, 37.09,
38.9, 36.26, 40.87, 38.13, 40.22, 38.48, 39.05, 36.91, 39.23, 39.37,
35.16, 39.05, 39.49, 39.61, 38.08, 36.62, 40.73, 40.09, 40.28, 38.15,
39.01, 40.9, 40.08, 37.17, 34.67, 32.89, 36.9, 37.15, 35.46, 39.07,
36.28, 36.37, 36.79, 33.67, 40.44, 36.22, 39.04, 39.32, 38.85, 40.42,
41.01, 38.8, 37.06, 38.39, 40.36, 41.44, 43.46, 35.68, 37.59, 37.85,
35.58, 38.73, 39.28, 38.41, 41.03, 35.68, 39.8, 42.17, 33.59, 40.82,
38.31, 37.7, 38.71, 37.53, 40.33, 36.93, 37.29, 38.9, 37.82, 42.99,
37.91, 36.81, 36.72, 39.92, 39.14, 39.39, 37.4, 37.18, 37.01, 38.56,
37.4, 35.99, 36.28, 34.98, 37.15, 40.78, 37.86, 39.64, 38.51, 38.73,
37.67]
```

```
plt.hist(sample_values2, bins=30, color='yellow', edgecolor='black')
plt.show()
```



#Checking sample1

```
sample_mean_value = np.mean(sample_mean_value)
print(sample_mean_value)
```

38.67603396603397

```
float(joined_df['Price'].mean())
```

38.6354

So Above you can see condition of central limit theorem is matching. So larger samples you take the more close it is to the average value of price

```
sample_values2 = np.mean(sample_values2)
print(sample_values2)
```

38.18

Every Hypothesis test is build on two statements

Null Hypothesis - It takes a default or initial assumption Its says no effect, no change. Alternative Hypothesis - The statement we will take or challenge. There is an effect or change.

Example 1: Medicine Effect

Null: The new medicine has no effect on blood pressure.

Alternative: The new medicine does reduce blood pressure.

Types of Error

Type 1 Error = You think something is there, but actually its not. Example: You say a patient has cancer, but really they dont.

Type 2 Error = You say nothing was there, but actually there was something(you missed it). Example: You say a patient doesn't have cancer, but really they do.

Example 1

Example: Fire Alarm System

Null Hypothesis (H_0): There is no fire.

Alternative Hypothesis (H_1): There is a fire.

Scenario 1

The alarm rings, but in reality there is no fire. Is this Type I or Type II?

Ans: Type 1 error because there was no fire but we assumed there was a fire.

Scenario 2

The alarm does not ring, but in reality there is a fire. Is this Type I or Type II?

Ans: Type2 error as there was really a fire but we said no fire, so we missed it, critical.

What is P Value?

p-value = how much chance the null hypothesis is right.

If p is small ($p \leq 0.05$) → Null is very unlikely → Reject H_0 → Accept alternative.

If p is big ($p > 0.05$) → Null is still possible → Keep H_0 → Do not accept alternative.

Example 1

Testing if average sales = ₹10,000.

$p = 0.02$ → very small → Null must go → Sales are not ₹10,000.

$p = 0.40$ → big → Keep Null → No strong evidence that sales differ from ₹10,000.

Example 2

Example: Patient Cancer Test

H_0 (Null): Patient does not have cancer.

H_1 (Alternative): Patient has cancer.

Case 1: The test gives a p-value = 0.01 (very small, less than 0.05).

Means: Null hypothesis is unlikely, accepts alternative.

Decision: Reject $H_0 \rightarrow$ Conclude the patient likely has cancer.

Case 2: The test gives a p-value = 0.40 (large, greater than 0.05).

Means: There's still a good chance that the null (no cancer) is true.

Decision: Fail to reject $H_0 \rightarrow$ Conclude the patient likely does not have cancer (or not enough evidence).

T test - One Sample

```
## T test - One Sample

import pandas as pd\

df = pd.read_csv(r'C:\Users\LENOVO\Downloads\Walmartsales
Dataset.csv')
df.shape

(1000, 18)

df.isnull().sum()

Invoice ID      0
Branch          0
City            0
Customer type   0
Gender          0
Product line    0
Unit price      0
Quantity        0
Tax 5%          0
Total          0
Date            0
Time            0
Payment         0
cogs            0
gross margin percentage  0
gross income    0
Rating          0
Customer ID     0
dtype: int64

df['Total']

0      548.9715
1      80.2200
```

```

2      340.5255
3      489.0480
4      634.3785
...
995     42.3675
996    1022.4900
997     33.4320
998     69.1110
999    649.2990
Name: Total, Length: 1000, dtype: float64

Sample = np.random.choice(df['Total'], size=30, replace=True)
print(Sample)

[203.175  570.78  520.8   149.3625 333.207  182.91  705.6315
209.769
 833.9625 255.15   57.1725 106.995  394.3275 714.3255 640.038
304.983
 614.943 484.89  580.419 829.08  494.76  122.472 180.4005
117.831
 86.247  92.442  93.0405 588.357  170.877  356.5485]
```

lets assume boss has given the sample and he asks please check if an average sale is Rs1000 or not. If real life scenario no one will give you a data

of 1 million and ask you to check They will only give you fraction of it to run the test

```

from scipy import stats

#boss assumption
mu = 1000

## null hypothesis = Rs 1000 is true
## alternative hypothesis = Rs 1000 is not true
t_stats, p_value = stats.ttest_1samp(Sample, mu)

t_stats = float(t_stats)
p_value = float(p_value)

print(f't_stats is {t_stats}')
print(f'p_value is {p_value}')

t_stats is -14.198119675272173
p_value is 1.3736209674498677e-14
```

P value is 0.0 so if p value is ≤ 0.05 null hypothesis is false
so we can say our boss average sale is not Rs1000